

Nama : Muhammad Abdul Fatah

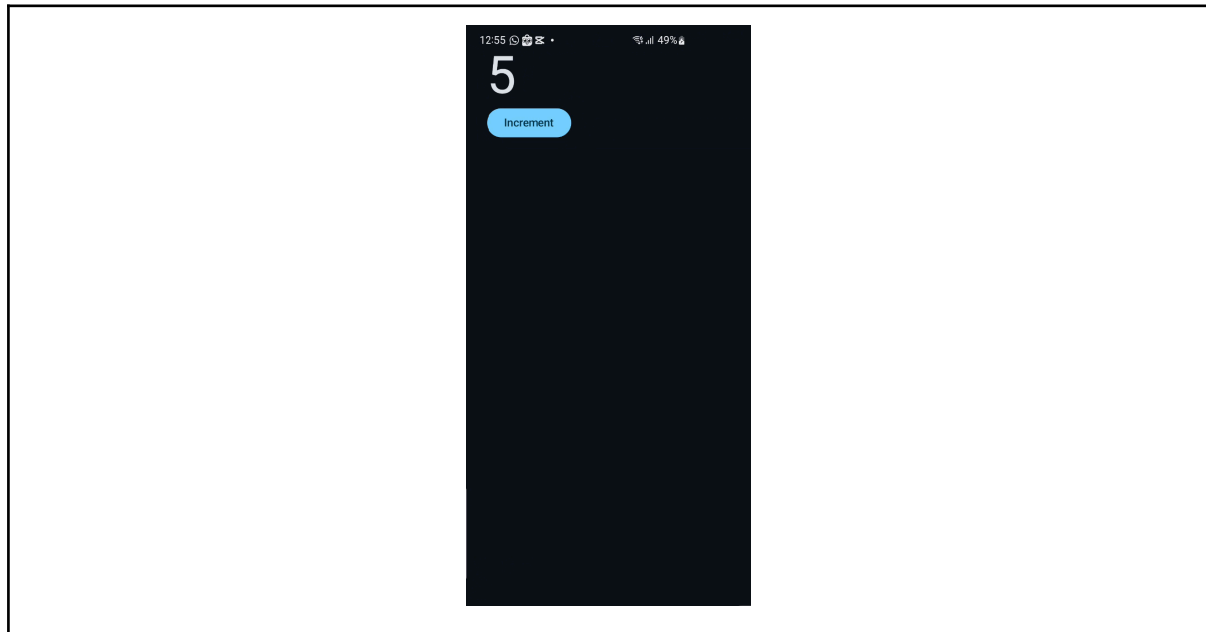
NIM : 245150407111090

Mata Kuliah : Pengembangan Aplikasi Mobile

Tugas Modul Praktikum 4

State Hoisting

Screenshot setelah tombol increment di klik beberapa kali

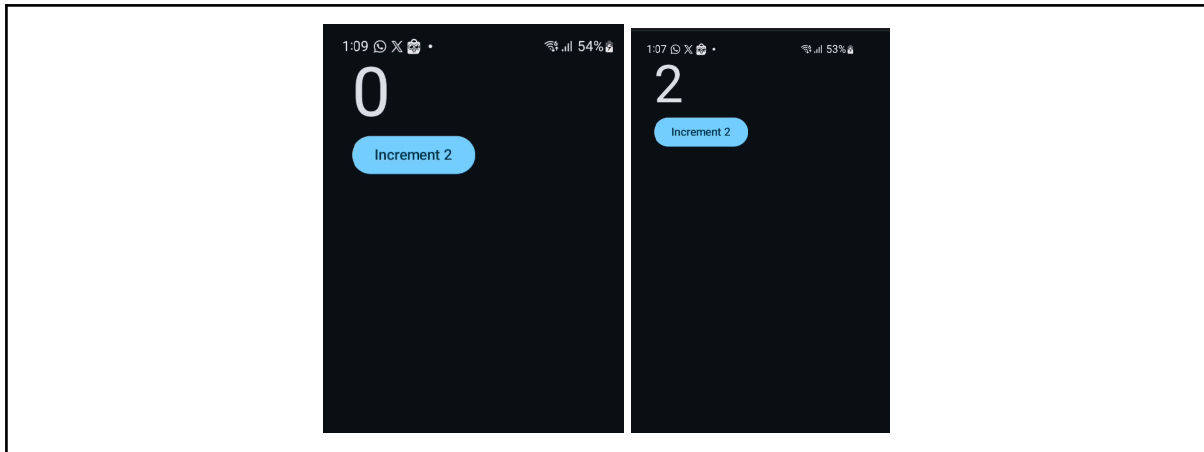


- a. Angkanya selalu nambah ketika tombol increment di klik
- b. Karena `onClick = {number++}` mengubah number yang bertipe `mutableStateOf`
- c. Angka
- d. Label yang nilainya "increment"

Nilai yang ditampilkan menjadi increment 2 dan labelnya juga diubah

```
val label by remember {mutableStateOf( value = "Increment 2")}
```

```
Button(onClick = {number+= 2}) {
```



Melakukan state dan event hoisting pada state dan event

```

fun CounterScreen(
    modifier: Modifier,
    number: Int,
    label: String,
    onClick: () -> Unit
) {
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = onClick) {
            Text(text = label)
        }
    }
}

Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
    var number by remember { mutableStateOf(value = 0) }
    CounterScreen(
        modifier = Modifier.padding(paddingValues = innerPadding).padding(all = 32),
        number = number,
        label = "Increment",
        onClick = { number++ }
    )
}

```

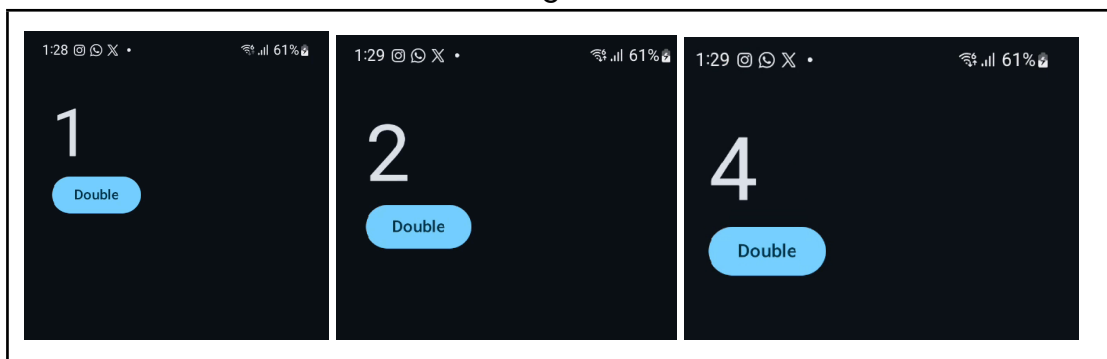
- Iya sama, yang berubah cuma lokasi state dari dalam composable
- Di onCreate, pada baris onClick = {number++}
- Dideklarasikan di onCreate() dalam setContent.

Label diubah jadi double, nilai awal variabel menjadi 1, number++ menjadi menggandakan nilai variabel number

```
setContent {  
    State_HoistingTheme {  
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
            var number by remember { mutableStateOf( value = 1) }  
            CounterScreen(  
                modifier = Modifier.padding( paddingValues = innerPadding).padding( all = 32.dp)  
                number = number,  
                label = "Double",  
                onClick = { number *= 2 }  
            )  
        }  
    }  
}
```

Jalankan program

a. Screenshot sebelum dan sesudah digandakan

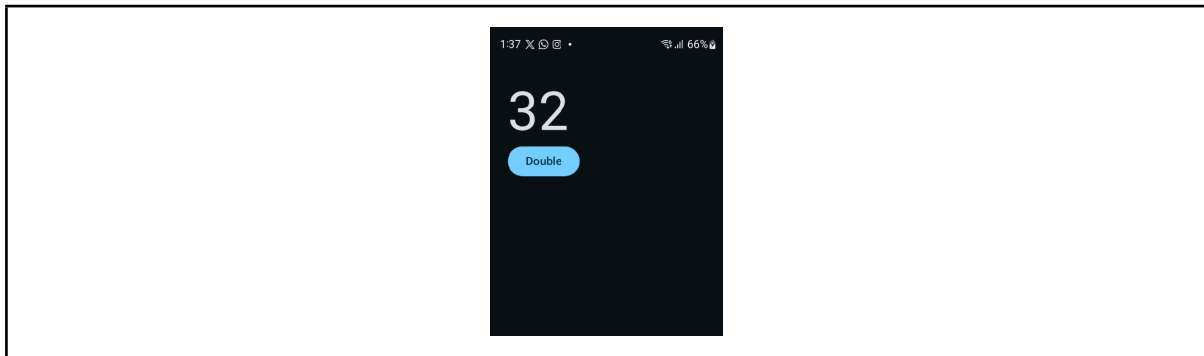


- b. Dilakukan di onCreate() pada deklarasi state number dan pemanggilan CounterScreen. Proses incrementnya lewat button onClick
- c. Tidak ada perubahan, yang berubah cuma dibagian onCreate aja

10. Karena perubahan dilakukan di pemanggil onCreate() tanpa mengubah counterscreen, dimana composablenya bersifat menampilkan data dan meneruskan event. Jadinya perubahan kebutuhannya cuma disatu tempat aja, sehingga counterscreen lebih reusable

Perubahan Konfigurasi

1. Screenshot tampilan antarmuka



2. Memutar posisi perangkat menjadi landscape

a.

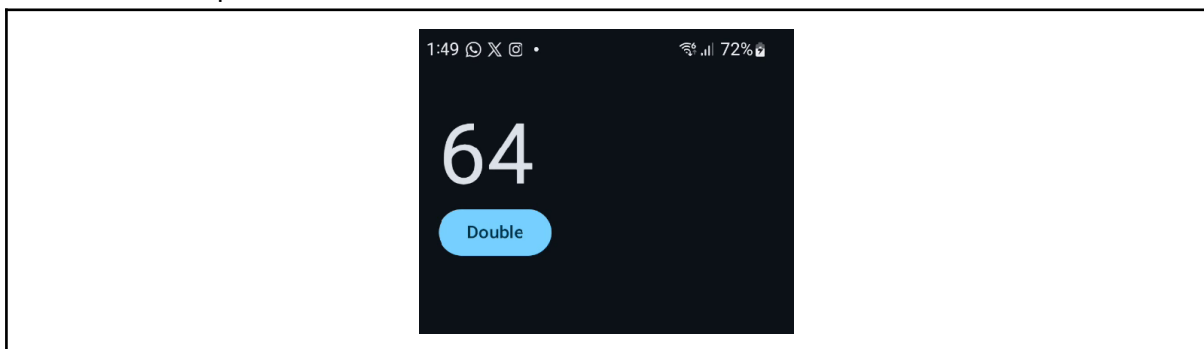


- b. Angkanya kembali menjadi 1 karena rotasi layar adalah perubahan konfigurasi dan android menghancurkan lalu membuat ulang activity. Dan remeber hanya menjaga nilai rekomposisi, tidak selama activity dibuat ulang

Merubah remember menjadi rememberSaveable

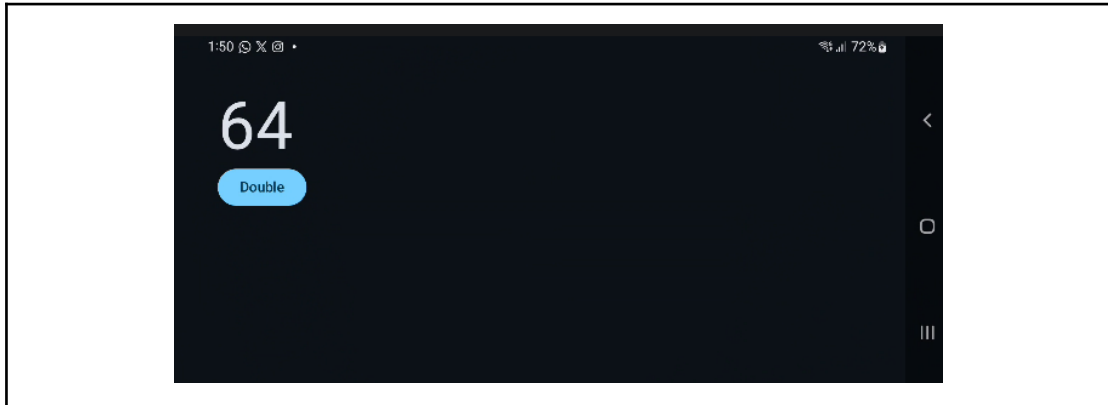
```
var number by rememberSaveable { mutableStateOf( value = 1) }
```

Screenshot tampilan antarmuka



3. Lakukan kembali perubahan konfigurasi

- a. Screenshot dalam posisi landscape



- b. Angkanya tetep 64 karena rememberSaveable menyimpan nilai state ke bundle dan memulihkannya setelah activity dibuat ulang
- c. Remember hanya mempertahankan state selama rekomposisi, sehingga state hilang jika terjadi perubahan konfigurasi. Sementara rememberSaveable, mempertahankan state saat activity dibuat ulang karena nilainya disimpan ke bundle

4. Contoh kasus nyatanya adalah isian form login atau register, terkadang pengguna udah mengetik email dan password abis itu memutar layarnya dan tetap ada.

Manfaatnya adalah agar data yang sudah diinput tidak hilang saat layar diputar, jadinya pengguna tidak perlu mengetik ulang dan experiencenya lebih baik

LaunchedEffect

Aktivitas Percobaan 1

1. Jalankan program

- Tampilan pertama kali adalah angka 1 dan snackbar bertuliskan "Number 1 is shown!" dibagian bawah
- Karena LaunchEffect otomatis dijalankan saat composable pertama kali masuk ke komposisi
- Pemicu pesan tersebut ada dikode program:

```
LaunchedEffect( key1 = number) {  
    snackbarHostState.showSnackBar( message = "Number $number is shown!")  
}
```

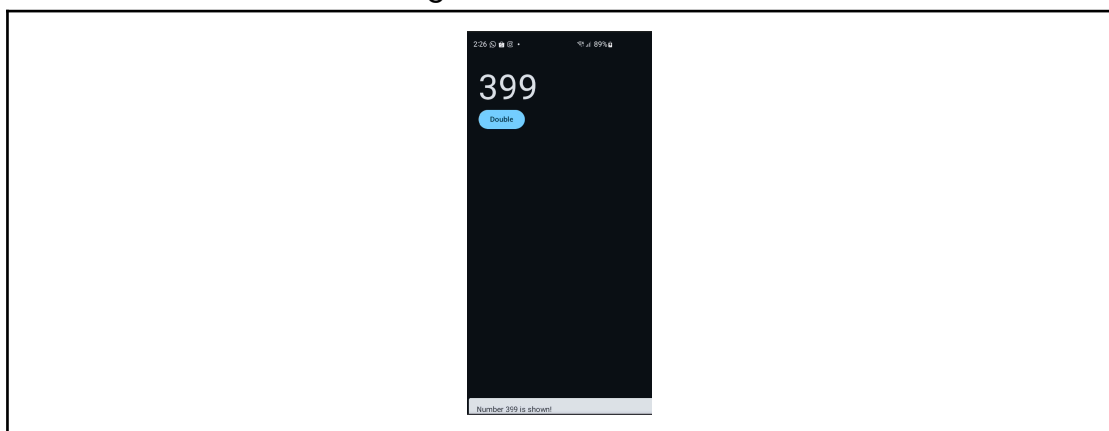
2. Klik Button "Double" satu kali

- Angka menjadi 2 dan snackbar bertuliskan "Number 2 is shown!" dibagian bawah
- Mengubah number 1 -> 2, dan number itu merupakan key dari LaunchEffect, saat key berubah maka compose meluncurkan ulang blok efeknya dengan key baru

Aktivitas Percobaan 2

1. Jalankan program

- Tampilan pertama kali adalah angka 1 dan snackbar bertuliskan "Number 1 is shown!" dibagian bawah
- Angka berubah menjadi angka acak yaitu 399 dan snackbar bertuliskan "Number 399 is shown!" dibagian bawah



- c. `LaunchEffect (Unit) {number = Repo.getData() }`, berjalan sekali dan mengisi number dengan angka acak. Perubahan number itu membuat `LaunchEffect (number)` berjalan ulang dan nampilin snackbar baru

2. Dari kedua percobaan dengan side effect dengan LaunchEffect

- a. `LaunchedEffect (state)` memakai state sebagai key dan efeknya dijalankan ulang setiap kali state itu berubah. Sementara `LaunchedEffect (Unit)` memakai key konstan yang tidak pernah berubah dan efeknya hanya dijalankan sekali
- b. `LaunchedEffect (state)` jalan ketika composable pertama kali masuk komposisi dan setiap kali nilai state berubah.
`LaunchedEffect (Unit)` jalan hanya sekali ketika composable pertama kali masuk komposisi